

Air Quality Sentinel Rollup

Sample task overview

Long-horizon Harbor coding environment · CPU ML engineering

This document covers what the task is, what the agent has to do, how it is graded, how frontier models performed and what ships in the folder. The task files and full rollout traces are included alongside it.

Task identification

Field	Overview
Task ID	airquality-sentinel-rollup
Category	CPU ML engineering: sensor data quality, hourly exceedance alerting and gated release operations
Scenario	A roadside benzene-alert service that returns fewer alerts than hours it was sent, and whose 24-hour CO display sometimes disagrees with the station's own log
Task format	Editable workspace, instruction, smoke tests, release-service CLI, protected release service, verifier, reference and alternate solutions
Real data	UCI Air Quality, roadside station hourly log (De Vito et al., 2008; CC BY 4.0)
Compute	2 vCPU, 7 GB RAM, no GPU; network available to the agent (harness requirement); not used by the task; grading offline
Agent budget	9,000 seconds; verifier 2,400 seconds
Agent harness	mini-swe-agent, used for both models' rollouts
Deliverable	A driven release trajectory ending in submit, and the frozen artifact the environment replays over 1,123 hidden hours the agent never sees

Description

The agent inherits an hourly alerting pipeline for a roadside air-quality station. Each hour a random-forest model calls whether the next hour will breach the benzene threshold, and the duty officer decides from that whether a public advisory goes out. Each alert is shown next to a 24-hour mean of the CO sensor. The engineer who built it has left, and two tickets came with the handover. In AQ-118 the control room gets back fewer alert rows than the hours it submitted. In AQ-124 an operator recomputed the 24-hour mean from the station log and got a different number on some alerts, but could not say which ones.

To solve it the agent has to do the **full engineer's job**: investigate the log and what the working tree computes from it, repair the pipeline across two files, and drive the release through a gated service: reproduction, deployment, a forced recovery from a faulty build, validation, promotion and submission. The service keeps its own state, ledger and packaged artifact out of the agent's reach, and grading runs the frozen artifact, not the working tree.

The core technical difficulty is **working out what the rolling mean is really averaging**. The logger writes a row every hour whatever the instrument is doing, so hours it did not measure are still in the file, carrying a marker value instead of a reading. The public UCI corpus is known for one marker, -200. This station log uses two: -200, and whole rows of zeros written while the analyser was swapped out. An agent that recalls -200 from memory and masks only that fixes most of the display but still averages zeros into every window that crosses a changeover. The protected validation never checks the rolling mean, so that release validates clean and fails at grading.

The incident the agent inherits

- **Hours silently withheld (AQ-118).** The serving path keeps only a legacy 06:00–21:59 publishing window from when the cabinet was powered down overnight, so 187 of the 562 hours in the current batch come back without a row. The job log records 375 hours alerted for 562 submitted, every week, with status ok.
- **Markers averaged as readings (AQ-124).** The 24-hour, 6-hour and delta rollups average whatever the logger wrote. The archive holds 71 hours marked -200 and 77 all-zero hours; the current batch holds 42 and 38. config.yaml pins the rule: the mean of the readings recorded in the 24 hours ending at that hour, to which an unmeasured hour contributes nothing.
- **Two places compute the display.** The model features are built in the serving path, but the number on the operator display is recomputed directly from the stitched station log in score_hours.py. Masking markers in serve.py alone does not reach the display, so the fix has to be in rollout.py as well.
- **A build that drops a config key.** Every deploy ships config.yaml without alerting.threshold. The first validate is therefore dirty; diagnostics.read names the stale_config family and the missing key, and a wrong family costs a recovery attempt and the diagnosis.

What the agent has to do

The work combines a **two-file repair** with an **ordered walk through the release service**. The reference trajectory takes 21 accepted actions, including a full rehearsal cycle before the fix; passing rollouts took 14–61. Repair can come before or after a first deployment cycle.

Phase	What happens	Operations
1. Investigation	Read the config, data card and job log; profile each channel; pull raw logger rows; compare what the working tree computes for an hour against the raw 24 hours behind it. Free and unlimited.	inspect, profile, sample_records, feature_audit
2. Repair	Serve every hour handed in. Treat both not-recorded markers as missing, so each window is a mean over the hours actually measured, and keep the smoke tests passing.	Edits to src/ (reference fix: rollout.py, serve.py)
3. Rehearsal and deployment	Re-run the backtest on the working tree, then package code and configuration (never data) into the bundle the service validates.	reproduce, deploy
4. Forced recovery	The first validate is dirty because the build dropped alerting.threshold. Read the diagnosis, apply that family, validate again on hidden hours.	validate, diagnostics.read, recovery.apply --family stale_config, validate

Phase	What happens	Operations
5. Promotion and submit	Promote the bundle (needs a clean validate after recovery), then submit, which freezes it as the graded artifact.	promote, submit

Validate returns only clean or dirty plus a coarse signal, never which hours. It is limited to four calls per session and checks contract, coverage and discrimination on a 561-hour sanity batch, not the rolling mean. Out-of-order operations are rejected without advancing, so the agent can correct and retry but cannot skip steps.

Example prompt

Opening of the task instruction given to the agent:

Roadside station 01 pushes an hourly exceedance alert to the control room. The model calls whether the next hour will breach the benzene threshold, and the duty officer decides from that whether a public advisory goes out. The engineer who built it has left. Two tickets came across with the handover and neither has an answer on it. ... AQ-124, the display disagrees: Each alert is shown beside a 24-hour mean of the CO sensor. An operator recomputed that mean from the station's own log for a handful of alerts and got a different number on some of them. ... What counts is the artifact frozen at submit and the trajectory that got it there, not a patch left sitting in your working tree and not a writeup.

The full instruction (task/instruction.md) specifies the ordered release path, the validate budget, the recovery rule and the scoring contract below.

Time, QA and difficulty calibration

Difficulty was measured, not asserted: each model was run four times (pass@4) inside the real task environment with the mini-swe-agent harness, and every attempt was graded by the same verifier.

Model	Strict passes	Mean reward	Note
Claude Opus 5 (xhigh reasoning requested)	3/4	0.75	36–54 steps per run, 8–16 minutes of agent time
Grok 4.7 (xhigh reasoning requested)	3/4	0.75	27–40 steps per run, 3–5 minutes of agent time

Measured profile: solvable, discriminating and consistent. Both frontier models solved 3 of 4 attempts, so the task is clearly solvable end to end without being saturated: a quarter of each model's attempts still shipped a display the station log does not support. The misses are not noise. Both are the same, precisely diagnosed failure (one attempt from each model handled only the -200 marker), which shows the task discriminates on exactly the skill it targets: checking assumptions about the data rather than recalling them.

Pass bars. At grading the environment runs the frozen artifact's scoring command over all 1,123 hidden terminal hours (24 January to 12 March 2005) and recomputes the display from the station log itself. Reward is 1 only if every clause holds:

Clause	Bar
Contract shape	timestamp, alert_probability, rollup_24h; one row per hour; probabilities in [0, 1]
Coverage of terminal hours	≥ 95%

Clause	Bar
Rollup agreement (1,099 graded hours, tolerance 10^{-6})	$\geq 99\%$
Discrimination (AUC against terminal labels)	≥ 0.85 (anti-degeneracy floor)
Artifact contents	Source and configuration only; no vendored data
Operational path	Eight gated steps in order to submit, with dirty validate → diagnosis → stale_config recovery → clean validate; ≥ 10 accepted actions, ≥ 2 observations

Per-attempt results (values that fail a bar in bold; — means the verifier stopped at the earlier clause):

Attempt	Result	Coverage	Rollup agreement	AUC	Accepted actions	Steps
Opus 5 · CaYme6k	Pass	100%	100%	0.9645	14	38
Opus 5 · PR65JvA	Pass	100%	100%	0.9498	26	54
Opus 5 · fbFVvYA	Pass	100%	100%	0.9498	15	39
Opus 5 · irorHko	Fail	100%	92.81%	—	14	36
Grok 4.7 · 2qiiPWj	Pass	100%	100%	0.9654	23	27
Grok 4.7 · WKX3X6u	Pass	100%	100%	0.9498	61	40
Grok 4.7 · cYWpz3u	Pass	100%	100%	0.9647	18	29
Grok 4.7 · ob53QZY	Fail	100%	92.81%	—	18	28

Steps and accepted actions measure different things. A step is one agent turn: a model call and the command it runs. An accepted action is one request the release service accepted, gated operations and free observations alike, as counted in the service's own log. Grok 4.7's WKX3X6u scripted the service and deployed 15 times, which is why its 61 accepted actions exceed its 40 steps.

All eight attempts reached 100% coverage and passed every operational check. The passing releases score AUC 0.95–0.97 against the hidden labels, well clear of the 0.85 floor: once the display and the hours are right, the fixed model discriminates as expected.

Why the attempts failed

Both failed attempts, **one from each model**, completed the whole release correctly. They removed the publishing-hours filter so every hour came back, diagnosed and cleared the stale_config fault, validated clean, promoted and submitted. Each failed on the rolling mean alone, at the same 92.81% agreement.

Opus 5 · irorHko counted the -200 marker in every channel at step 6, then wrote a sentinel mask for values at or below -200 into the shared log reader. It never counted zeros. **Grok 4.7 · ob53QZY** did print that 30 published CO readings were exactly zero, but masked only the -200 sentinel it had already identified as the UCI marker. In both, every window that crosses an analyser changeover still averages hours of zeros. This is the failure the folklore negative control isolates (only -200 handled, pinned at 90.99% agreement for its exact form).

That is by design. Validation certifies contract, coverage and discrimination only, so a display that is still wrong is certified clean. The six passing attempts found the second marker by looking at the data rather than recalling the corpus: each counted exact zeros in the log and found them in whole rows and contiguous

blocks, matching the station note that the analyser was swapped with the logger left running. The pass rate therefore reflects whether the agent checks its assumptions about the data, not whether it can follow the release checklist.

Grading design

Reward is binary (1 only if every component passes), but the verifier reports each component separately, so every attempt yields a dense diagnostic record rather than a single bit.

Component	Overview
Technical result	The frozen artifact's scripts/score_hours.py is run by the environment over all 1,123 hidden terminal hours. The verifier recomputes the rolling mean from the station log with both markers excluded and checks contract, coverage, rollup agreement on the 1,099 hours past the first window, and AUC.
Robustness	The opening 24 hours are not graded, because how an implementation seeds the first window across the archive gap is a convention the contract does not pin. A row-based and a time-based 24-hour window both score 1, and so do percentile-ranked probabilities.
Operational path	The accepted operational actions in the root-owned ledger must contain reproduce, deploy, validate, diagnostics.read, recovery.apply, validate, promote and submit in order, with a clean validate after recovery, ending at submit.
Recovery	The injected build fault must be seen dirty, diagnosed and repaired with the family the diagnosis named (stale_config).
Multi-file repair	The display is computed in rollup.py and the withheld hours come from serve.py; a control that fixes only serve.py scores 0 on the rollup clause.
Isolation	Candidate code runs as a separate unprivileged user without access to the hidden batches, labels or service state. The bundle may carry only source and configuration. Workspace logs, stdout claims and forged reward files are ignored.

Failure, retries and limits

How we determine the agent has failed

Grading is deterministic and strictly binary: reward is **1.0 (pass)** or **0.0 (fail)**. The verifier reads environment-owned state the agent cannot write: the root-owned action ledger and service state, and the frozen artifact the service copied at submit, rerun by the environment on hours the agent never saw, not the live workspace.

Reward is 1.0 **only if every condition holds**: the frozen artifact meets the contract, coverage, rollup and discrimination bars on the hidden terminal hours, carries no data, the ordered walk was driven to submit, and the injected build fault was diagnosed and cleared with the stale_config family. Miss any one and the reward is 0.0: a display value the log does not support, withheld hours, a skipped or reordered step, a wrong recovery, or no submit.

Retries

- **No free retries of the outcome.** Within an attempt the agent can correct and retry: an out-of-order operation is rejected without advancing, and the agent can redeploy after changing the pipeline.
- **Feedback is lossy and rationed.** Validate is limited to four calls for the whole session and returns only clean or dirty plus a coarse signal. It never checks the rolling mean, so it cannot be used to search for the right display.
- **A wrong recovery is not free.** Each deploy gets two recovery attempts; a wrong family burns one and discards the diagnosis. Redeploying resets the attempts but reapplies the build fault.
- **Across attempts:** pass@4 means four fully independent attempts, each in a fresh container. They are independent samples used to measure the solve rate, not retries the agent controls. This task measured 3 of 4 for both Claude Opus 5 and Grok 4.7.

Step and time limits

- **Time limit:** a hard 9,000 seconds (2 h 30 min) per attempt. An attempt still running at the limit is stopped.
- **Step limit:** none. The mini-swe-agent harness enforces no per-attempt step cap, so an attempt runs until the agent stops or reaches the time limit.
- **Never submitting is a fail, not an error.** An attempt that never submits, or times out first, is graded 0, since there is no frozen artifact to grade. Genuine infrastructure failures, such as a container or harness fault, are reported separately as inconclusive and are never recorded as 0.

QA process

The task passes the same automated QA pipeline as the rest of the pack. Evidence for this task ships in the rollout folders:

- **Build and gates.** The Docker image built cleanly in both evaluation runs (digests sha256:4f43aeb1... and sha256:28f655dc..., recorded in each gold/gates.md). In both runs the reference solution scores 1 and the untouched starting state scores 0.
- **Build-time guards.** The build fails if solution or verifier material, hidden batches or answer-like strings appear in the workspace, or if the hidden and probe directories are not root-only.
- **Three structurally different positive solutions.** Masking both markers before a rolling mean over the full stitched log (oracle), a rolling sum over a rolling count of measured hours with a bounded archive lead (alternate 1), and markers cleaned in every channel at read time with a different bounded lead (alternate 2) each score 1. Two further positive controls, a time-based 24-hour window and percentile-ranked probabilities, also score 1, so values are graded rather than the shape of one fix.
- **Sixteen negative controls, all required to score 0 for a predicted reason:** untouched environment; full protocol with no fix; serving fixed but markers still averaged (passes smoke tests and a clean validate); rollup fixed but hours still withheld; only serve.py changed; only -200 handled (90.99% agreement); both markers but a window over the last 24 readings instead of 24 hours (86.44%); correct fix by the shortest path; fault never diagnosed; guessed recovery family; forged stdout and reward files; hunting for labels; raw-sensor ranking with no model; a vendored rollup table; and emptied or binary-noise modules.
- **Determinism checks.** A perturbation harness re-runs the gold fix reformatted, with different numeric dtypes and literal styles, and requires exactly 1.0 each time; the calibration harness runs every control in a fresh environment and checks the reason as well as the reward.

Environment and execution

Python 3.12 CPU container (numpy 2.4.2, pandas 2.3.3, scikit-learn 1.7.2, scipy 1.17.0, PyYAML 6.0.3) with 10 GB of storage; network available to the agent (harness requirement); not used by the task; grading offline. The agent works unprivileged in /workspace/target/: config.yaml, the station archive and current batch, a data card, five pipeline modules (channels, features, rollup, model, serve), scoring and backtest scripts, smoke tests, the job log and env_cli.py. The release service runs as root behind a Unix socket, owns all release state and runs candidate code as a separate unprivileged user.

The runtime contract is a single command, scripts/score_hours.py --batch <csv> --out <csv>, writing one row per hour handed to it with timestamp, alert_probability and rollup_24h.

Raw base data

Field	Overview
Source	UCI Machine Learning Repository, Air Quality (dataset 360); De Vito et al. (2008)
License	Creative Commons Attribution 4.0 International
Content	Real hourly records from a roadside station in an Italian city, March 2004 to April 2005: five metal-oxide sensor channels, temperature and humidity, and a reference benzene analyser
Integrity	Rows are split by time; none is synthesised, re-weighted or altered. The only edit is that alternate runs of the logger's not-recorded hours are marked with all-zero rows instead of -200
Visible data	6,549-hour archive with the reference analyser (March–December 2004); 562-hour current batch without it (December 2004)
Held-out batches	Validate 561 hours (January 2005) and terminal 1,123 hours (24 January to 12 March 2005); label: next-hour benzene above 14.0 (about 12% of terminal hours)

Technical domains / relevant benchmarks

The task combines MLE-bench-style ML engineering (sensor data quality, missing-value semantics, rolling features for a production classifier) with SWE-bench and DeepSWE-style multi-file repository repair and Terminal-Bench-style autonomous terminal execution. Its gated operations, rate-limited one-bit validation, hidden service state and injected build fault also exercise T-bench-style stateful tool use, applied to a production ML release rather than a simulated conversation.

Folder contents

Folder	Contents
task/	The complete Harbor task: instruction.md, task.toml, environment (Dockerfile, release service, hidden batches, grading probe, workspace), tests (verifier and probe shim), solution (reference and two alternate solutions) and _private (workspace generator, calibration and perturbation harnesses and the negative-control battery with its expected outcomes).

Folder	Contents
Opus 5 pass@4/	agg/ holds the aggregate report and every trial's trajectory and verifier output for the four Claude Opus 5 rollouts; gold/ holds the build log, image digest and the oracle and no-op gate results.
Grok 4.7 pass@4/	The same layout for the four Grok 4.7 rollouts.

How the task was created

The real-data problem was chosen first: a public sensor corpus whose logger records unmeasured hours as values, which makes a mean of what was recorded easy to state and easy to get subtly wrong. The incident was built around it: the eras were cut so outages fall inside the graded hours, a second marker was introduced so that recalling the dataset's known sentinel is not enough, the display was computed in a second place so the fix has to span two files, and validation was limited to shape and discrimination. Each of these is shown to matter by a negative control that fails on it. The task was then packaged with three structurally different solutions, the control battery, root-only verification and build-time guards.

Acknowledgement

I confirm that I have reviewed the tasks and believe they are realistic, technically substantive, clearly specified, and appropriately graded. They reflect genuine professional workflows and are not low-effort, artificial, or "slop" tasks.

Name: Suleiman Mahmood

Role: Co-Founder

Signature: /s/Suleiman Mahmood

Date: 9.28.26
